Coupling and Cohesion

Software Engineering Design Principles

1 Introduction

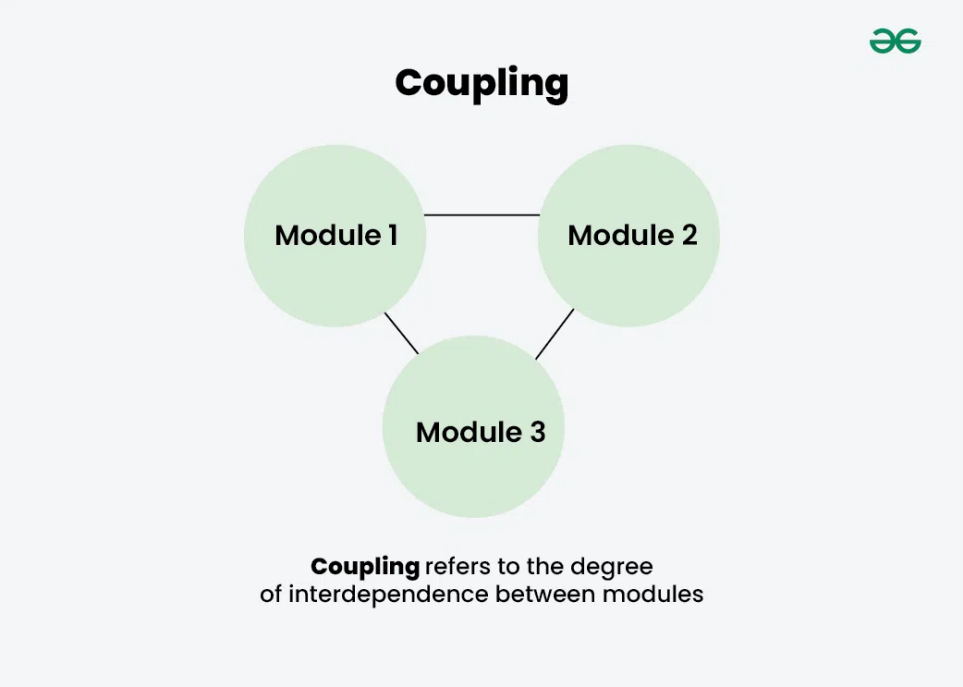
The purpose of the **Design Phase** in the Software Development Life Cycle (SDLC) is to produce a solution based on the Software Requirement Specification (SRS). The output of this phase is the **Software Design Document (SDD)**.

Coupling and **Cohesion** are two fundamental metrics used to evaluate the quality of a software design.

2 What is Coupling and Cohesion?

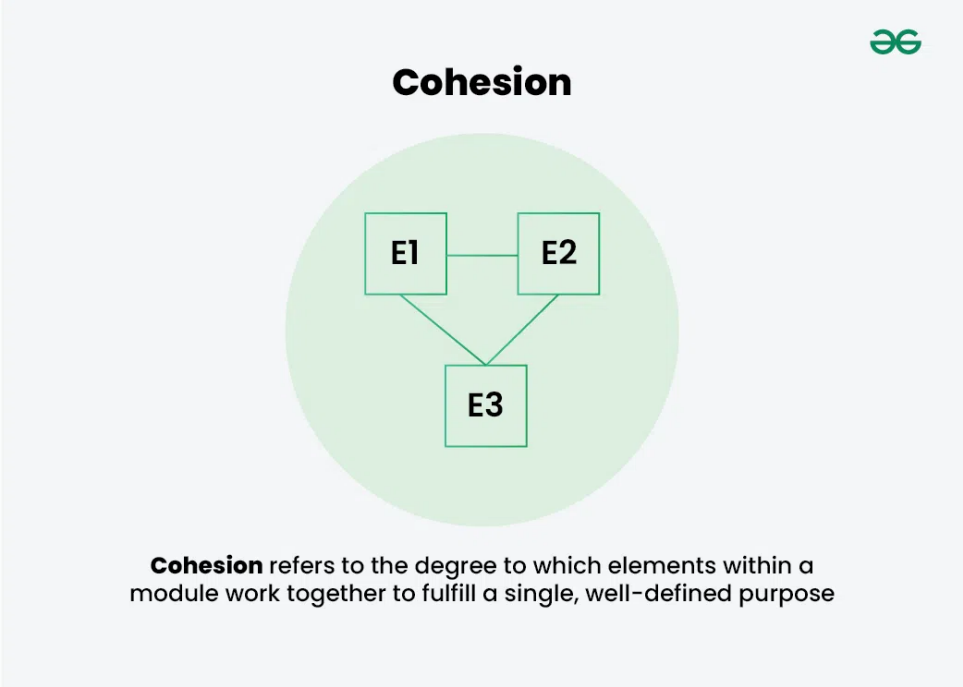
Coupling refers to the degree of interdependence between software modules.

- **High coupling**: Modules are tightly connected
- **Low coupling**: Modules are independent and loosely connected



Cohesion refers to the degree to which elements within a module are related and focused on a single purpose.

- **High cohesion**: Elements are strongly related
- **Low cohesion**: Elements serve unrelated purposes



Good software design aims for **low coupling** and **high cohesion**.

OOP Link (SRP): In object-oriented design, **high cohesion** is closely related to the **Single Responsibility Principle (SRP)**: a class/module should have only **one responsibility** (one primary reason to change).
High cohesion often goes together with **loose coupling** because focused classes typically require fewer dependencies.

3 Design Process Overview

Software design is an iterative two-part process:

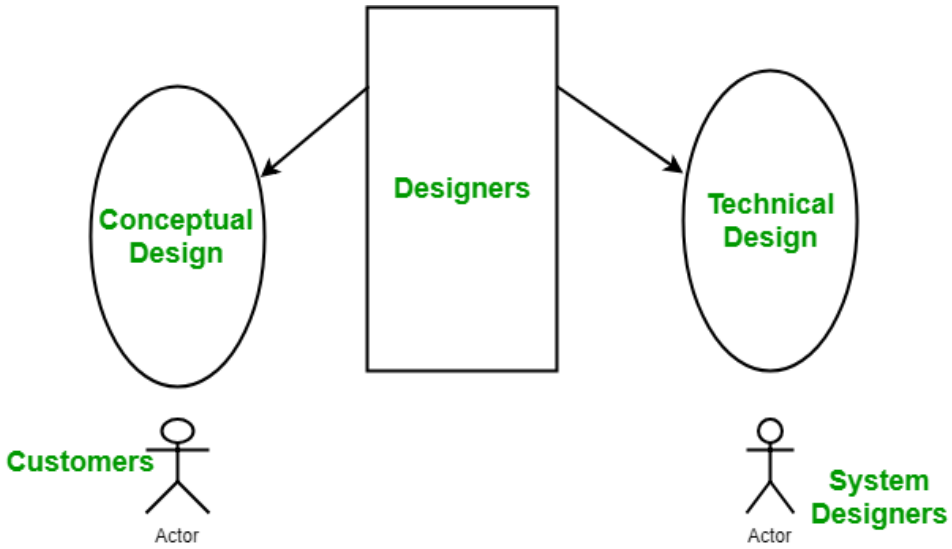
3.1 Conceptual Design

- Written in customer-understandable language
- Detailed explanation of system characteristics
- Describes system functionality
- Independent of implementation
- Closely linked with SRS (Software Requirements Specification)

3.2 Technical Design

- Hardware components
- Software architecture and hierarchy

- Data structures and data flow
- Network and I/O design
- Interfaces between components



4 Modularization

Modularization is the process of dividing a system into independent modules, each performing a specific task.

- Improves system understanding
- Simplifies maintenance
- Enables reuse of modules

5 Types of Coupling

Coupling measures how strongly modules depend on each other.

Tight vs Loose Coupling in OOP

Tight coupling occurs when a class depends directly on a *specific implementation* of another class. **Loose coupling** is achieved when classes depend on *abstractions* (e.g., interfaces) rather than concrete implementations.

Tight coupling example (idea): A `Car` class directly creates an `Engine` object (hard-coded dependency).
Loose coupling improvement: Make `Car` depend on an `IEngine` interface (or inject the engine dependency).

Coupling Types (Best to Worst)

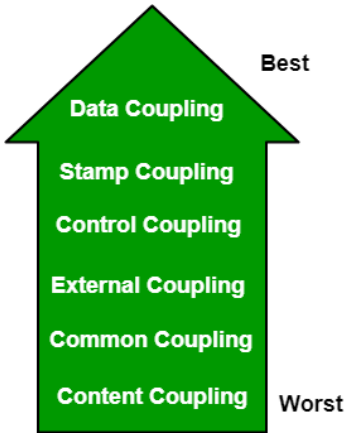


Figure 1: Comparison of best and worst coupling

- **Data Coupling** – Modules communicate by passing only the required data.
- **Stamp Coupling** – Entire data structures are passed between modules.
- **Control Coupling** – Control information is passed to influence module behavior.
- **External Coupling** – Dependence on external systems, files, or hardware.
- **Common Coupling** – Modules share global data.
- **Content Coupling** – One module directly accesses or modifies another (worst).
- **Temporal Coupling** – Modules depend on execution timing or order.
- **Sequential Coupling** – Output of one module becomes input of another.
- **Communicational Coupling** – Modules share a common communication medium.
- **Functional Coupling** – One module directly relies on another's functionality.
- **Data-Structured Coupling** – Modules share common data structures.
- **Interaction Coupling** – Methods of one class invoke methods of another.
- **Component Coupling** – One class contains or uses another class as a component.

Content coupling should always be avoided.

OOP Practices to Achieve Loose Coupling

- Depend on **interfaces/abstractions** rather than concrete classes.

- Use **Dependency Injection** instead of creating dependencies inside a class.
- Apply patterns like **Strategy**, **Adapter**, or **Facade** to reduce direct dependencies.

6 Types of Cohesion

Cohesion measures how closely related the responsibilities of a module are.

Cohesion Types (High to Low)

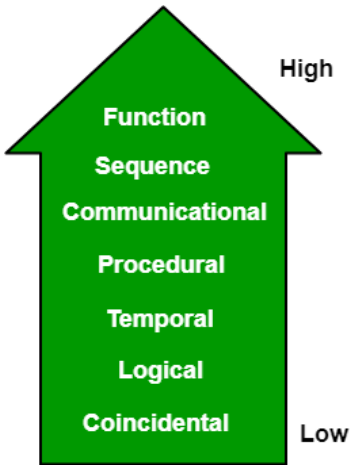


Figure 2: Comparison of high and low cohesion

- **Functional Cohesion** – All elements perform a single well-defined task (best).
- **Layer Cohesion** – Tasks grouped by abstraction or responsibility level.
- **Informational Cohesion** – Tasks operate on the same data structure.
- **Sequential Cohesion** – Output of one task becomes input of another.
- **Communicational Cohesion** – Tasks operate on the same data or output.
- **Procedural Cohesion** – Tasks related by execution order.
- **Temporal Cohesion** – Tasks related by timing (e.g., initialization).
- **Logical Cohesion** – Tasks grouped by logical similarity.
- **Coincidental Cohesion** – Unrelated tasks grouped together (worst).

Functional cohesion is the most desirable form of cohesion.

OOP Example (Low Cohesion → High Cohesion)
A low-cohesion class often mixes unrelated responsibilities. Example idea: a single `StudentRecord` class that stores data, calculates GPA, prints reports, and sends email notifications.
Refactor: split responsibilities into focused classes such as `Student`, `GradeCalculator`, `ReportPrinter`, and `EmailNotifier`. This improves cohesion and usually reduces coupling.

7 Advantages of Low Coupling

- Improved maintainability
- Better modularity
- Easier testing
- Enhanced scalability

8 Advantages of High Cohesion

- Improved readability
- Better error isolation
- Increased reliability
- Easier reuse

9 Disadvantages of High Coupling

- Increased complexity
- Reduced flexibility
- Difficult maintenance
- Poor reusability

10 Disadvantages of Low Cohesion

- Code duplication
- Unclear module purpose
- Reduced maintainability
- Higher chance of errors

11 Conclusion

A well-designed software system should exhibit:

- **Low coupling** between modules
- **High cohesion** within modules

Following these principles results in software that is:

- Easier to maintain
- More reliable
- Highly reusable
- Scalable and adaptable